PROJET DE REMPLACEMENT DE STAGE

Conception d'un Mini-SOC à empreinte réduite pour PME Togolaises

Moteur de corrélation stateful en Python
et évaluation comparative avec Wazuh

Année Académique 2025-2026

Auteur :

- **KPODONOU K. Gaël**
  L3 Informatique et Systèmes

Encadrement :

Directeur accompagnateur :

. . . . . .

Matière / UE :

. . . . . .

École Polytechnique de Lomé
Université de Lomé

1. Contexte et problématique

Le Togo, en progression dans l'indice mondial de cybersécurité (UIT, 2024), dispose d'une Agence Nationale de Cybersécurité (ANCy) mais d'un tissu de PME aux budgets très limités. Une licence Splunk dépasse 10 000 $/an, un EDR professionnel coûte 50–150 $/poste/an, des montants structurellement incompatibles avec les capacités locales. Parallèlement, on constate une pénurie de compétences opérationnelles et une faible couverture des menaces propres à l'Afrique de l'Ouest (phishing SMS/WhatsApp, fraude Mobile Money, ransomwares sur Windows non patchés).

Question centrale : Est-il possible de construire un système de détection d'intrusion *stateful*, capable d'identifier des chaînes d'attaques complexes, avec des contraintes matérielles compatibles avec les PME togolaises (≤ 8 Go RAM, coût logiciel nul), et dont les performances de détection sont comparables à celles de Wazuh sur les mêmes scénarios ?

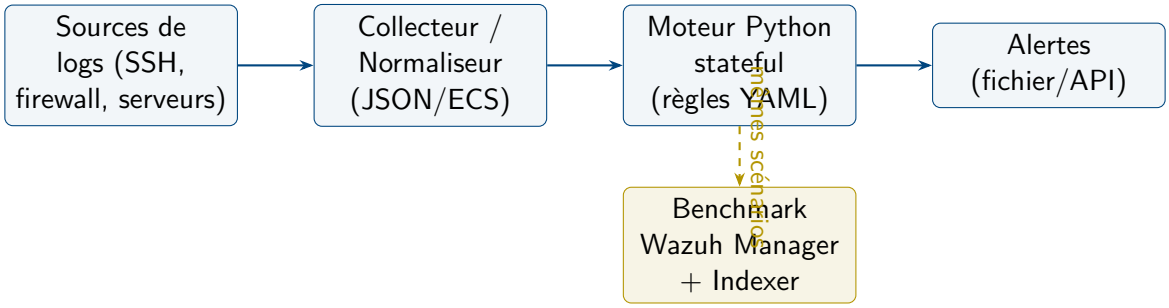
2. Hypothèses de travail

H1. Équivalence de détection – Taux de vrais positifs (TPR) $\geq 90\%$, taux de faux positifs (FPR) $\leq 10\%$, sans différence significative avec Wazuh.

H2. Supériorité en ressources – Consommation RAM < 4 Go en charge, latence de détection $< 3\,000$ ms.

H3. Adaptabilité locale – Règles de corrélation modifiables par un développeur Python intermédiaire, sans expertise SIEM.

3. Architecture résumée



Le laboratoire est entièrement virtualisé (VMs OPNsense/Debian/Kali, conteneurs Docker pour waeuh) et isolé du réseau physique. La mémoire totale du lab n'excède pas 3,5 Go pour les phases de développement, 7,5 Go lors du benchmark Wazuh, laissant une marge confortable sur un hôte à 16 Go.

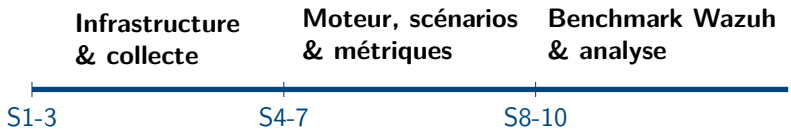
4. Scénarios d'attaque retenus

Scénario	Règle de corrélation (résumé naturel)
1 – Brute-force SSH	Détection d'au moins 10 échecs SSH depuis une même IP en 60 s, suivis d'une connexion réussie depuis la même IP dans les 120 s.
2 – Reconnaissance + Exfiltration	Scan réseau vers ≥ 20 hôtes en 30 s depuis une IP interne, puis transfert sortant ≥ 10 Mo vers une IP externe dans les 300 s.

5. Métriques d'évaluation (objectifs de conception)

Métrique	Cible moteur Python	Comparaison Wazuh
TPR (sensibilité)	$\geq 90\%$	À mesurer (objectif H1)
FPR	$\leq 10\%$	À mesurer (objectif H1)
Latence de détection	$< 3\,000$ ms	À mesurer (objectif H2)
RAM en charge	< 4 Go	À mesurer (objectif H2)

6. Planning macro (10 semaines)



Semaines 1–3 : topologie, IaC, pipeline de logs, isolation du dataset d'évaluation. Semaines 4–7 : moteur Python, tests unitaires, simulation des deux scénarios, premières mesures. Semaines 8–10 : déploiement Wazuh, benchmark sur le dataset isolé, analyse comparative et soutenance.

7. Impact local et perspectives

— **Coût logiciel nul** : stack 100% open-source (Python, Docker, Alpine, Wazuh), compatible matériel de seconde main.

— **Alignement national** : projet en phase avec la Stratégie Nationale de Cybersécurité du Togo 2024–2028.

— **Règles contextualisables** : possibilité d'étendre la détection à d'autres menaces locales (fraude Mobile Money, phishing WhatsApp).

Rapport complet de 16 pages (architecture, benchmark) disponible pour la réunion de cadrage.
Dépôt Git : Le dépôt gitlab officiel du projet ici –
`https://gitlab.com/AdrienStuart/nyx/-/blob/master/docs/mini-soc-rapport.pdf`